

Proyecto: Consola Retro de Videojuegos
Módulo: Mednafen y mapeo del control DualSense vía bridge uinput

Martínez Jiménez Alejandro (Núm. cuenta: 319130865)

Equipo: Silva Castro F.J. (319266399) Arellanes Conde E. (319322743)

Fundamentos de Sistemas Embebidos 3 de junio de 2026

1. Objetivo

Integrar el emulador multi-sistema Mednafen sobre una imagen Linux generada con Buildroot para Raspberry Pi 4, configurarlo para ejecutar ROMs de NES, SNES, Game Boy Advance y Game Boy a pantalla completa con audio HDMI, y resolver el problema de que Mednafen no reconoce el gamepad DualSense de manera nativa sobre SDL2+KMSDRM, mediante el desarrollo de un *bridge* userspace que traduce eventos de joystick a teclado virtual usando el subsistema uinput del kernel Linux.

2. Introducción

2.1. Emulación de videoconsolas

Un emulador es un programa que reproduce en software el comportamiento del hardware de una máquina distinta. Para emular una consola retro como la NES debe simular su CPU (en este caso el MOS 6502), su chip de gráficos (PPU), su chip de sonido (APU) y los buses internos, leyendo el contenido del cartucho original como una imagen binaria denominada *ROM* [1].

2.2. Mednafen

Mednafen (*My Emulator Doesn't Need A Frickin' Excellent Name*) es un emulador multi-sistema libre, distribuido bajo licencia GPLv2, que se ejecuta exclusivamente desde la línea de comandos sin interfaz gráfica propia [1]. Su diseño es modular: cada sistema emulado constituye un *módulo* (núcleo) independiente, seleccionable con el parámetro `-force_module` [2]. Los módulos relevantes para este proyecto son `nes`, `snestfaust`, `gba` y `gb`.

La salida de video y el audio se delegan a **SDL2** (Simple DirectMedia Layer) [3], lo cual le permite operar tanto sobre un servidor X11 como directamente sobre el framebuffer del kernel mediante el backend KMS/DRM (Kernel Mode Setting / Direct Rendering Manager) [4, 5]. La configuración del emulador, incluyendo el mapeo de controles, se almacena en un archivo de texto plano en `$HOME/.mednafen/mednafen.cfg` [2].

2.3. Gamepad DualSense y el subsistema HID de Linux

El gamepad DualSense de PlayStation 5 (VendorID 0x054c, ProductID 0x0ce6) cuenta con soporte oficial en el kernel Linux desde la versión 5.12 mediante el driver `hid_playstation`, publicado por Sony Interactive Entertainment [6, 7]. Este driver expone el control como un dispositivo HID estándar, accesible tanto a través de la API tradicional `/dev/input/jsX` (joydev) como a través de la API moderna `/dev/input/eventX` (evdev).

2.4. El problema técnico abordado

A pesar de que el kernel Linux reconoce correctamente el DualSense, Mednafen 1.32.1 al operar sobre SDL2 con backend KMSDRM presenta un problema documentado: su parser de configuración

rechaza el formato de identificación de joystick generado por SDL2 para este modelo de gamepad, mostrando el error `Bad joystick bn string` al intentar cargar el archivo `mednafen.cfg`. La solución implementada en este proyecto consiste en construir un puente userspace que lee el DualSense vía `/dev/input/js0`, crea un teclado virtual mediante el subsistema `uinput` del kernel, y traduce cada botón del gamepad a una pulsación de tecla. Mednafen interpreta el teclado virtual como un dispositivo de entrada nativo y responde como si el usuario estuviera tecleando.

3. Lista de materiales

Tabla 1: Materiales para el módulo de emulación y control

Componente	Especificación / equivalente	Cant.
Raspberry Pi 4 Model B	4 GB de RAM (mín. 2 GB)	1
Tarjeta microSD	Clase 10, mínimo 16 GB	1
Fuente de alimentación	USB-C, 5 V / 3 A certificada	1
Monitor	HDMI nativo o VGA con adaptador HDMI–VGA activo	1
Cable microHDMI–HDMI	Estándar 1.4 o superior	1
Gamepad DualSense (PS5)	Equivalente: cualquier gamepad con <code>hid_playstation</code> o HID genérico	1
Memoria USB	Para importación de ROMs adicionales	1
ROMs legales	Formatos <code>.nes</code> , <code>.smc</code> , <code>.gba</code> , <code>.gb</code>	varias
PC anfitriona Linux	Debian/Ubuntu/Kali para compilar Buildroot	1
Software		
Buildroot 2025.02	Framework de cross-compilación [9]	–
Mednafen 1.32.1	Emulador multi-sistema [1]	–
SDL2 (backend kmsdrm)	Capa multimedia [3]	–
ALSA	Subsistema de audio del kernel	–
eudev + libevdev	Detección dinámica de dispositivos [8]	–

4. Descripción de los componentes relevantes

4.1. Subsistema `uinput` del kernel Linux

`uinput` es un módulo del kernel que permite a un proceso de espacio de usuario crear dispositivos de entrada virtuales. Al escribir sobre el nodo `/dev/uinput`, una aplicación puede declarar las capacidades del dispositivo (teclas que soporta, ejes, botones), crear el dispositivo con `ioctl(fd, UI_DEV_CREATE)`, y a partir de ese momento inyectar eventos (`EV_KEY`, `EV_SYN`) que el kernel entregará al resto del sistema como si provinieran de hardware físico. La Tabla 2 resume los `ioctl` relevantes.

Tabla 2: ioctls principales del subsistema uinput

ioctl	Función
UI_SET_EVBIT	Habilita un tipo de evento (e.g. EV_KEY)
UI_SET_KEYBIT	Habilita una tecla específica (e.g. KEY_A)
UI_SET_PROPBIT	Marca propiedades del dispositivo (e.g. INPUT_PROP_DIRECT)
UI_DEV_SETUP	Configura nombre, VendorID, ProductID y bus
UI_DEV_CREATE	Crea el nodo /dev/input/eventX virtual
UI_DEV_DESTROY	Destruye el dispositivo virtual al terminar

4.2. Driver hid_playstation y APIs de joystick

El kernel Linux expone el DualSense vía dos interfaces simultáneas:

- /dev/input/js0 — API heredada (joydev). Lectura por bloques de estructuras struct js_event con campos type, number y value. Es la interfaz usada por jstest y por nuestro bridge.
- /dev/input/event0 (o similar) — API moderna (evdev). Lectura por estructuras struct input_event. Preferida por SDL2 cuando la variable SDL_INPUT_LINUXEV=1 está definida.

4.3. SDL2 sobre KMS/DRM

SDL2 sobre Linux puede operar sin servidor gráfico mediante su backend kmsdrm [4], escribiendo directamente al framebuffer del kernel sin requerir X11 ni Wayland [5]. Esto reduce la latencia y elimina la dependencia de un compositor de ventanas. Las variables de entorno relevantes para Mednafen se resumen en la Tabla 3.

5. Advertencias de seguridad y cuidado de los componentes

- **Antiestática:** sostener la Raspberry Pi por sus bordes. El uso de pulsera antiestática reduce el riesgo de daño permanente por descargas electrostáticas sobre los circuitos integrados.
- **Temperatura:** durante la emulación, la CPU alcanza entre 40% y 70% de carga. Instalar disipadores y mantener la ventilación adecuada.
- **Memorias microSD:** grabar siempre con el comando sync al final para asegurar que los buffers de escritura se vacían a la tarjeta antes de extraerla.
- **Acceso a /dev/uinput:** el nodo requiere permisos de escritura. El proyecto define reglas udev para conceder permiso de lectura/escritura a los grupos pertinentes sin requerir privilegios de superusuario en tiempo de ejecución.

6. Configuración previa: paquete Mednafen en Buildroot

La generación de la imagen base con Buildroot [9]. Para este módulo solamente se requiere verificar tres elementos en make menuconfig:

```

1 Target packages -> Games -> [*] mednafen
2 Target packages -> System tools -> [*] eudev
3                               [*] eudev: enable rule generator
4 Target packages -> Hardware handling -> [*] libevdev
```

Listing 1: Activaciones requeridas en menuconfig

La activación de `evdev` [8] y `libevdev` es **crítica**: sin estas bibliotecas, SDL2 en modo KMSDRM es incapaz de detectar el teclado virtual que el bridge crea con `uinput`. Este detalle no aparece documentado en la mayoría de tutoriales de Mednafen sobre Raspberry Pi y fue una de las dificultades técnicas centrales del proyecto.

7. Wrapper de Mednafen

Mednafen requiere un conjunto específico de variables de entorno para operar sobre SDL2 + KMSDRM [4]. Estas variables se concentran en un wrapper de shell que el frontend invoca en lugar de llamar al binario directamente. El wrapper también espera a que el bridge esté listo antes de lanzar el emulador y redirige toda la salida estándar a `/dev/null` para mantener la pantalla limpia.

```

1 #!/bin/sh
2 # Espera a que el gamepad-bridge cree el teclado virtual
3 for i in 1 2 3 4 5; do
4     [ -f /tmp/gamepad-bridge-ready ] && break
5     sleep 1
6 done
7
8 export SDL_INPUT_LINUXEV=1
9 export SDL_JOYSTICK_DISABLE_LINUX_HOTPLUG=0
10 export SDL_VIDEODRIVER=kmsdrm
11 export SDL_KMSDRM_REQUIRE_DRM_MASTER=0
12 export HOME=/home/pi
13 export MEDNAFEN_ALLOWMULTI=1
14
15 exec /usr/bin/mednafen "$@" > /dev/null 2>&1

```

Listing 2: `/usr/local/bin/mednafen-wrapper.sh`

Tabla 3: Variables de entorno de SDL2 utilizadas por el wrapper [3, 4]

Variable	Propósito
<code>SDL_VIDEODRIVER=kmsdrm</code>	Renderizado directo al framebuffer del kernel
<code>SDL_INPUT_LINUXEV=1</code>	Fuerza a SDL2 a leer dispositivos vía <code>evdev</code>
<code>SDL_KMSDRM_REQUIRE_DRM_MASTER=0</code>	Permite acceso al framebuffer sin ser proceso “DRM master” [4]
<code>HOME=/home/pi</code>	Ruta donde Mednafen busca <code>.mednafen/mednafen.cfg</code> [2]
<code>MEDNAFEN_ALLOWMULTI=1</code>	Permite múltiples instancias del emulador

8. Bridge `uinput`: traducción DualSense a teclado virtual

Este es el componente central del módulo de control. El bridge es un programa en C de aproximadamente 180 líneas que se ejecuta en segundo plano desde el `inittab` y permanece activo durante toda la sesión.

8.1. Diseño general

1. Abrir `/dev/input/js0` en modo no bloqueante para leer eventos del DualSense.
2. Abrir `/dev/uinput` y configurar las capacidades del teclado virtual (EV_KEY, conjunto de teclas habilitadas, VendorID/ProductID).
3. Crear el dispositivo virtual con `UI_DEV_CREATE`.
4. Crear el archivo bandera `/tmp/gamepad-bridge-ready` para que el wrapper de Mednafen sepa que puede lanzarse.
5. Entrar al bucle principal: leer cada `js_event`, traducir el botón o eje a la tecla equivalente, e inyectar la pulsación con `write(ufd, &ie, sizeof(ie))` seguido de un evento `EV_SYN/SYN_REPORT`.

8.2. Mapeo de botones

El mapeo se determinó analizando con `jstest` la numeración que el driver `hid_playstation` asigna a cada botón físico del DualSense [6], y eligiendo teclas que coincidieran con los *defaults* de Mednafen para cada sistema, descubiertos mediante `grep` sobre el `mednafen.cfg` generado tras una sesión de mapeo con la combinación `Alt+Shift+1` integrada en el emulador [2].

Tabla 4: Mapeo DualSense → tecla virtual → función NES

Botón DualSense	Tecla virtual emitida	Función NES
Cruz (X)	KEY_A (cód. 4)	Botón A (saltar)
Cuadrado	KEY_F (cód. 9)	Botón B (correr)
Triángulo	KEY_Z	Extra
Círculo	KEY_X	Extra
L1	KEY_Q	Hombro izq. (SNES)
R1	KEY_E	Hombro der. (SNES)
Options	KEY_ENTER (cód. 40)	Start
Share	KEY_RIGHTSHIFT (cód. 229)	Select
Botón PS	KEY_F12	Salir de Mednafen
D-pad izq./der.	KEY_LEFT / KEY_RIGHT	Cruceta horizontal
D-pad arr./aba.	KEY_UP / KEY_DOWN	Cruceta vertical

8.3. Fragmento clave del bridge

```
1 #include <linux/uinput.h>
2 #include <linux/joystick.h>
3
4 int ufd = open("/dev/uinput", O_WRONLY | O_NONBLOCK);
5
6 /* Habilitar tipos de evento y teclas */
7 ioctl(ufd, UI_SET_EVBIT, EV_KEY);
8 ioctl(ufd, UI_SET_EVBIT, EV_SYN);
9 ioctl(ufd, UI_SET_PROPBIT, INPUT_PROP_DIRECT);
10
11 int keys[] = { KEY_A, KEY_F, KEY_Z, KEY_X, KEY_Q, KEY_E,
12               KEY_ENTER, KEY_RIGHTSHIFT, KEY_F12,
13               KEY_UP, KEY_DOWN, KEY_LEFT, KEY_RIGHT };
```

```

14 for (size_t i = 0; i < sizeof(keys)/sizeof(*keys); i++)
15     ioctl(ufd, UI_SET_KEYBIT, keys[i]);
16
17 /* Identidad del dispositivo virtual */
18 struct uinput_setup usetup = {0};
19 usetup.id.bustype = BUS_USB;
20 usetup.id.vendor = 0x413c;
21 usetup.id.product = 0x2003;
22 strcpy(usetup.name, "Dell USB Keyboard Bridge");
23 ioctl(ufd, UI_DEV_SETUP, &usetup);
24 ioctl(ufd, UI_DEV_CREATE);
25
26 /* Bucle principal: leer js_event, emitir tecla */
27 struct js_event je;
28 while (running) {
29     if (read(jfd, &je, sizeof(je)) != sizeof(je)) { usleep(5000); continue; }
30     je.type &= ~JS_EVENT_INIT;
31     if (je.type == JS_EVENT_BUTTON) {
32         int key = btn_to_key(je.number); /* tabla de mapeo */
33         if (key) emit_key(ufd, key, je.value ? 1 : 0);
34     } else if (je.type == JS_EVENT_AXIS) {
35         if (je.number == 6) handle_axis(ufd, &axis_x_state, je.value,
36                                         KEY_LEFT, KEY_RIGHT);
37         else if (je.number == 7) handle_axis(ufd, &axis_y_state, je.value,
38                                             KEY_UP, KEY_DOWN);
39     }
40 }

```

Listing 3: Creación del teclado virtual y bucle principal (gamepad-bridge.c)

8.4. Manejo de ejes del D-pad

El D-pad del DualSense se reporta como dos ejes (no como botones digitales): eje 6 horizontal y eje 7 vertical, ambos con valores en el rango $[-32767, +32767]$. Para evitar pulsaciones espurias por ruido cerca del centro, el bridge mantiene una máquina de estados de tres posiciones (centro, negativo, positivo) y solo emite press o release cuando el estado cambia, usando un umbral de ± 16000 .

9. Lanzamiento de la imagen y prueba del control

9.1. Grabación de la imagen

```

1 sudo umount /dev/mmcblk0p* 2>/dev/null
2 sudo dd if=output/images/sdcard.img of=/dev/mmcblk0 \
3     bs=4M status=progress conv=fsync oflag=sync
4 sync

```

Listing 4: Grabar la imagen generada en la microSD

9.2. Verificación del flujo completo

Al insertar la microSD en la Raspberry Pi y suministrar alimentación, el sistema realiza la siguiente secuencia:

1. El kernel arranca silenciosamente (`quiet loglevel=0` en `cmdline.txt`).
2. `eudev` [8] se inicializa y aplica las reglas `udev` de `/etc/udev/rules.d/99-input.rules`.
3. `inittab` lanza `gamepad-bridge` en `tty4`, que crea el teclado virtual y marca `/tmp/gamepad-bridge-ready`.
4. `inittab` lanza el frontend en `tty1`, que muestra la galería de ROMs.
5. Al pulsar Cruz (X) en el DualSense, el frontend invoca `mednafen-wrapper.sh -force_module nes /home/pi/roms/<juego>.nes`.
6. `Mednafen` lee `mednafen.cfg`, interpreta el teclado virtual del bridge como dispositivo de entrada y responde a cada pulsación del DualSense traducida.
7. Al presionar el botón PS del DualSense, el bridge emite `KEY_F12` y `Mednafen` termina limpiamente.
8. El respawn del `inittab` relanza el frontend automáticamente.

10. Otros componentes del sistema (desarrollados por el equipo)

A continuación se resumen brevemente los componentes implementados por los otros integrantes del equipo, cuya integración fue necesaria para que el módulo de emulación funcionara dentro de un sistema completo. Los detalles correspondientes se encuentran en los reportes individuales de cada autor.

10.1. Splash de arranque

Para reemplazar el logotipo de Buildroot por una pantalla de bienvenida con imagen y audio, se desarrolló un paquete propio llamado `retroconsole-splash`. Este consiste en un binario en C que decodifica un archivo PNG con **libpng** y lo vuelca directamente al framebuffer del kernel (`/dev/fb0`), centrando la imagen y soportando los formatos RGB565 y RGBA32. El audio del splash se reproduce en paralelo con `aplay` de ALSA. El init script `S05splash` se ejecuta antes que el frontend, mostrando la imagen durante tres segundos al boot. El mismo binario es reutilizado por el daemon de importación USB para mostrar el mensaje “COPIANDO ROMs”.

10.2. Frontend de galería

El frontend es una aplicación en C++17 de aproximadamente 580 líneas que renderiza la galería de juegos usando **SDL2** y **SDL2_ttf** [3]. Sus funciones principales son: escanear recursivamente `/home/pi/roms` agrupando los archivos por sistema según su extensión, presentar la lista con etiquetas de color por consola, recibir entrada del DualSense vía la API `SDL_GameController`, y lanzar el wrapper de `Mednafen` mediante `fork()` + `exec1()` cuando el usuario pulsa Cruz. Al retornar del emulador, el frontend abre el subsistema de joystick con `SDL_QuitSubSystem` seguido de `SDL_InitSubSystem` y reescanea los dispositivos, restaurando el control completo en la galería. Adicionalmente, el frontend muestra una etiqueta amarilla “NUEVO” junto a las ROMs recién importadas, que desaparece cuando el cursor se posa sobre el juego.

10.3. Importador USB automático

El importador es un daemon en Python que arranca al boot mediante el init script `S95usbimporter`. Cuando detecta una nueva partición, el daemon termina los procesos de `Mednafen` y del frontend para liberar el framebuffer, muestra el splash “COPIANDO ROMs” invocando al binario `retroconsole-splash`, monta la USB probando los sistemas de archivos `vfat`, `exfat`, `ext4` y

ntfs, y copia recursivamente las ROMs con extensiones .nes, .smc, .sfc, .gba, .gb y .gbc a /home/pi/roms/, sobrescribiendo si ya existe. Al copiar cada archivo, crea un marcador <rom>.new junto al archivo, que el frontend interpreta para mostrar la etiqueta “NUEVO”. Tras desmontar la USB y cerrar el splash, el daemon emite la bandera REFRESH_FLAG, y el respawn del inittab relanza el frontend con la lista actualizada de juegos.

11. Análisis de rendimiento

Tabla 5: Rendimiento medido en Raspberry Pi 4 (1.8 GHz, sin overclocking)

Sistema	FPS obj.	FPS medido	CPU (%)	Núcleo
NES	60	60	≈10	nes
SNES	60	60	≈35	snes_faust
SNES bsnes	60	55–58	≈72	snes
Game Boy	60	60	≈8	gb
GBA	60	60	≈40	gba

El módulo `snes_faust` (*High-Level Emulation*) ofrece una emulación menos precisa que `snes` (*cycle-accurate*, basado en `bsnes`) pero a un costo computacional sustancialmente menor, lo cual es preferible en hardware ARM [2]. El bridge introduce una latencia adicional medida de aproximadamente 1 ms entre la pulsación física y la entrega del evento al emulador, imperceptible al usuario.

12. Resultados

- Mednafen opera a pantalla completa sobre KMS/DRM sin servidor X11 [4].
- El DualSense controla la consola en su totalidad (frontend, emulador y salida), sin necesidad de teclado.
- NES, Game Boy y GBA emulan a 60 FPS estables; SNES con `snes_faust` mantiene 60 FPS.
- El bridge `uinput` resuelve la incompatibilidad de Mednafen con el formato de joystick de SDL2 para el DualSense.
- Al salir del emulador con el botón PS, el frontend reabre el subsistema de joystick mediante `SDL_QuitSubSystem + SDL_InitSubSystem` y reescanea los controles, lo cual restaura todos los botones (Triángulo, hombros, etc.) en el menú.

13. Conclusiones

La integración de Mednafen sobre una imagen Linux generada con Buildroot, sin escritorio ni servidor gráfico, es técnicamente viable y eficiente. El backend KMS/DRM de SDL2 elimina la latencia asociada a compositores de ventanas [4, 5], y el módulo `snes_faust` ofrece la mejor relación rendimiento/precisión para SNES sobre ARM [2].

El principal hallazgo técnico fue que Mednafen no acepta el formato de identificación de joystick que SDL2 genera para el DualSense bajo KMSDRM. En lugar de modificar el código fuente de Mednafen, se optó por una solución más portable: el subsistema `uinput` del kernel Linux permite crear dispositivos de entrada virtuales desde espacio de usuario; al exponer el DualSense como un teclado USB virtual con identidad propia, Mednafen lo trata como hardware nativo. Esta estrategia es transparente, eficiente y reutilizable para cualquier emulador o aplicación que tenga problemas

similares con un gamepad específico.

14. Cuestionario

1. **¿Por qué Mednafen no puede leer el DualSense directamente bajo SDL2+KMSDRM, si el kernel lo expone correctamente?**

Mednafen serializa la identificación del joystick en su archivo de configuración usando un GUID derivado del formato interno de SDL2. Para el DualSense, este GUID contiene caracteres que el parser de `mednafen.cfg` rechaza con el error `Bad joystick bn string`, impidiendo cargar el mapeo guardado. Construir un puente `uinput` elude el problema porque el dispositivo virtual aparece como un teclado USB con `VendorID/ProductID` asignados por nosotros, formato que el parser acepta sin objeciones.

2. **¿Qué ventajas tiene el subsistema `uinput` respecto a parchear el código fuente de Mednafen?**

`uinput` es parte del kernel Linux desde hace más de una década y su API es estable. Modificar Mednafen requeriría mantener un parche local que habría que reaplicar con cada actualización del emulador. El bridge en cambio funciona contra cualquier versión del emulador y, además, es reutilizable: la misma lógica permite usar el DualSense con cualquier programa que solo soporte teclado.

3. **¿Por qué se requirió activar `eudev` en Buildroot para que el bridge funcionara?**

SDL2 con backend KMSDRM descubre dispositivos de entrada consultando `libudev` en tiempo de ejecución [4]. Sin un servicio `udev` activo, los nodos `/dev/input/event*` existen en el filesystem pero SDL2 no los enumera y por lo tanto no detecta el teclado virtual creado por el bridge. Activar `eudev`, un *fork* de `udev` de `systemd` independiente del sistema de `init` [8], y `libevdev` resuelve la enumeración.

4. **¿Por qué el D-pad del DualSense se reporta como ejes y no como botones, y cómo se traduce?**

El driver `hid_playstation` expone el D-pad como dos ejes analógicos virtuales (eje 6 horizontal, eje 7 vertical) con valores discretos `-32767`, `0` o `+32767`, replicando el comportamiento que SDL2 espera de un *game controller* mapeado [6]. El bridge implementa una máquina de estados de tres posiciones por eje y emite `KEY_LEFT/KEY_RIGHT/KEY_UP/KEY_DOWN` solo en las transiciones, lo que evita pulsaciones repetidas por ruido cerca del cero y produce el mismo efecto que una cruceta digital.

5. **¿Qué papel juega la combinación `Alt+Shift+1` integrada en Mednafen, y cómo se aprovechó para descubrir el mapeo?**

Mednafen incluye un modo asistido de reconfiguración de controles que se activa con `Alt+Shift+1` [2]. Al usarlo dentro de un juego NES pulsando los botones del DualSense en el orden solicitado por el asistente, Mednafen escribió en `mednafen.cfg` los códigos de tecla que estaba recibiendo del teclado virtual. Inspeccionando el archivo después se obtuvieron los códigos exactos (`KEY_A=4`, `KEY_F=9`, `KEY_ENTER=40`, etc.) que Mednafen esperaba, y se ajustó el mapeo del bridge para coincidir, fijando la configuración de manera permanente sin necesidad de volver a usar `Alt+Shift+1` en el futuro.

15. Recursos del proyecto

Repositorio GitHub: https://github.com/Alextrenee/ProyectoFinal_AMFconsole

Video demostrativo: https://youtu.be/Je9JPb_1E3o

Referencias

- [1] Mednafen Project, “Mednafen: Multi-system Emulator — Official site,” 2025. [Online]. Available: <https://mednafen.github.io/>. [Accessed: Jun. 2026].
- [2] Mednafen Project, “Mednafen General Documentation,” 2025. [Online]. Available: <https://mednafen.github.io/documentation/>. [Accessed: Jun. 2026].
- [3] SDL Development Team, “SDL2 — Simple DirectMedia Layer Wiki,” 2024. [Online]. Available: <https://wiki.libsdl.org/SDL2/FrontPage>. [Accessed: Jun. 2026].
- [4] SDL Development Team, “SDL_HINT_KMSDRM_REQUIRE_DRM_MASTER — SDL2 Wiki,” 2024. [Online]. Available: https://wiki.libsdl.org/SDL2/SDL_HINT_KMSDRM_REQUIRE_DRM_MASTER. [Accessed: Jun. 2026].
- [5] The Linux Kernel Organization, “Kernel Mode Setting (KMS),” *Linux Kernel Documentation*, 2025. [Online]. Available: <https://www.kernel.org/doc/html/latest/gpu/drm-kms.html>. [Accessed: Jun. 2026].
- [6] R. Colenbrander, “hid-playstation: HID driver for the PlayStation 5 DualSense controller,” *Linux kernel source tree, drivers/hid/hid-playstation.c*, Sony Interactive Entertainment, 2020. [Online]. Available: <https://github.com/torvalds/linux/blob/master/drivers/hid/hid-playstation.c>. [Accessed: Jun. 2026].
- [7] M. Larabel, “Sony Publishes An Official Linux Driver For PlayStation 5 DualSense Controllers,” *Phoronix*, Dec. 2020. [Online]. Available: <https://www.phoronix.com/news/Sony-HID-PlayStation-PS5>. [Accessed: Jun. 2026].
- [8] eudev Project, “eudev — Standalone dynamic device naming support,” GitHub repository, 2024. [Online]. Available: <https://github.com/eudev-project/eudev>. [Accessed: Jun. 2026].
- [9] Buildroot Development Team, “Buildroot: Making Embedded Linux Easy,” 2025. [Online]. Available: <https://buildroot.org/downloads/manual/manual.html>. [Accessed: Jun. 2026].